

# Manuale meccanico SQL + DAO per prove TdP

Database Chinook e pattern ricorrenti da prove reali / simulazioni GitHub

Obiettivo: darti un criterio rapido e operativo per leggere una consegna d'esame, decidere quali query scrivere nel DAO, quali dataclass creare, e quali parti lasciare al Model. Il manuale è pensato per essere consultato durante l'esame: cerca le parole chiave del testo e copia il pattern corrispondente.

## Regola madre

Prima individua sempre: dropdown, nodi, dato del nodo, condizione di arco, peso, verso, dettagli richiesti, ricorsione. La query corretta nasce da questi sette elementi, non dal codice.

## 0. Fonti e casi usati come riferimento

Il manuale integra i facsimili Chinook svolti insieme, le due simulazioni Chinook appena viste, e i repository GitHub pubblici che hai elencato. I nomi sotto servono come mappa mentale dei pattern ricorrenti.

| Caso / repository      | Tipo grafo                     | Pattern DAO dominante                                                                                       |
|------------------------|--------------------------------|-------------------------------------------------------------------------------------------------------------|
| 2024-07-04-A UFO       | Orientato non pesato           | Self-join su stessa tabella: stesso stato, data precedente -> data successiva. Arco(id_passato, id_futuro). |
| 2024-07-18-A Genes     | Orientato pesato               | Nodi filtrati da range; archi da interactions + classification; verso da attributo del nodo/relazione.      |
| 2025-11-10 Bike orders | Orientato pesato temporale     | Archi temporali con DATEDIFF; peso calcolabile in SQL o con dizionario quantità.                            |
| 2025-09-15 F1 drivers  | Non orientato pesato           | Self-join results su stessa gara e stesso constructor; COUNT(*) come peso.                                  |
| 2025-07-10 Products    | Orientato pesato               | Nodi prodotti; vendite per prodotto in dizionario; coppie grezze in DAO; verso e peso nel Model.            |
| 2025-06-26 F1 circuits | Non orientato pesato complesso | Nodi tutti i circuiti + dettagli per nodo; coppie circuiti grezze; peso da dizionario finiti.               |
| Lab11 Chinook artisti  | Orientato pesato               | Generi; nodi artisti; popolarità dict; archi grezzi cliente-artista in comune.                              |
| Lab12 attori           | Non orientato pesato           | DAO estrae dati grezzi sporchi; parsing peso nel Model.                                                     |
| Baseball               | Non orientato completo pesato  | Nodi squadre; salario dict; grafo completo nel Model.                                                       |
| ArtsMia                | Non orientato pesato           | Self-join tabella associazione mostra-oggetto; COUNT(*) come peso.                                          |
| Lab10 countries        | Non orientato non pesato       | Archi confine: year <= anno, contype=1, state1 > state2.                                                    |
| Lab09 airports         | Non orientato pesato           | Rotte aggregate con LEAST/GREATEST e AVG(DISTANCE).                                                         |

## 1. Come leggere il testo: algoritmo in 90 secondi

Ogni traccia va trasformata in una lista di decisioni. Non partire mai dal DAO. Parti da queste domande.

- Quali dropdown devo popolare? Ogni dropdown richiede quasi sempre una query DISTINCT dedicata.
- Chi sono i nodi? Se il testo dice "i vertici sono...", quella è la query getAllNodes.
- I nodi sono tutti, anche isolati, oppure solo quelli che soddisfano un filtro? Attenzione: spesso il testo dice "tutti i prodotti della categoria" anche se non venduti; questi nodi restano isolati.
- Il nodo deve salvarsi dati aggiuntivi? Esempi: circuito con piazzamenti, MediaType con dizionario mese -> acquisti, Composer con lista brani validi.
- Quando esiste un arco? Cerca parole come: stessa invoice, stessa playlist, stesso cliente, stesso mese, stesso team, stesso stato, distanza temporale, co-presenza.
- Il peso è direttamente un conteggio/media/somma per coppia? Se sì, il DAO può restituire Arco(id1, id2, peso).

- Il verso dipende da una data? Allora SQL può già orientare. Il verso dipende da un valore del nodo? Di solito meglio Model.
- Il punto 2 chiede ricorsione? Il DAO non c'entra quasi mai: tutto parte dal grafo già costruito nel Model.

#### Errore tipico da evitare

Non confondere condizione di arco e peso. Esempio MediaType nello stesso mese: la condizione e condividere almeno una coppia (CustomerId, mese), ma il peso e vendite1 + vendite2. Sono due dati diversi.

## 2. Regola DAO / Model: dove mettere cosa

Il DAO deve estrarre dati dal database in strutture semplici. Il Model deve costruire il grafo, calcolare algoritmi e gestire confronti fra nodi quando il confronto e più leggibile in Python.

| Situazione nel testo                                          | Scelta consigliata                                                              | Perché                                                                                             |
|---------------------------------------------------------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Arco = co-presenza con peso COUNT DISTINCT facile             | DAO.getAllEdges() -> Arco(id1,id2,peso)                                         | SQL e forte su GROUP BY, COUNT, AVG, SUM. Model resta pulito.                                      |
| Arco = relazione grezza, ma verso/peso dipende da valore nodo | DAO.getAllEdges() -> Arco(id1,id2);<br>DAO.getValori(); Model decide verso/peso | Eviti query enormi e rendi chiaro il confronto.                                                    |
| Arco = confronto tra set/lista salvati nel nodo               | DAO carica nodi + dati; Model crea archi                                        | Quando il testo parla di dizionari per nodo, e spesso più chiaro usare set.intersection nel Model. |
| Arco = temporale vecchio -> nuovo                             | SQL con data1 < data2 oppure Model su nodi ordinati                             | Se il peso si calcola bene in SQL usa DAO; se vuoi chiarezza, ordina nel Model.                    |
| Peso sporco o stringa da pulire                               | DAO restituisce dato grezzo; Model pulisce                                      | Esempio Lab12: incassi testuali, valori nulli, simboli valuta.                                     |
| Grafo completo                                                | DAO nodi + dizionario valori; Model doppio ciclo completo                       | Non serve query archi: tutti con tutti.                                                            |

## 3. Quando creare la dataclass Arco

### 3.1 Arco(id1, id2, peso): quando il peso si calcola bene in SQL

Usala quando la query può restituire direttamente la coppia e il peso con GROUP BY. E il caso più comodo per grafi non orientati pesati e per alcuni grafi orientati temporali.

```
from dataclasses import dataclass

@dataclass
class Arco:
    id1: int
    id2: int
    peso: float
```

Parole del testo che ti spingono verso questa scelta:

- "Il peso è il numero di ..."
- "Il peso è la media di ..."
- "Il peso è la somma di ..."
- "Due nodi sono collegati se compaiono nella stessa ..."
- "Il peso è COUNT DISTINCT di entità comuni"

Esempi: ArtsMia oggetti nella stessa exhibition, Lab09 aeroporti con distanza media, F1 drivers compagni di squadra, Composer nelle stesse playlist, Album nella stessa invoice.

```

SELECT x1.idNodo AS id1,
       x2.idNodo AS id2,
       COUNT(DISTINCT relazione.comune) AS peso
FROM   tabella_relazione r1,
       tabella_relazione r2,
       tabella_nodo x1,
       tabella_nodo x2
WHERE  ...
       AND r1.comune = r2.comune
       AND x1.idNodo < x2.idNodo
GROUP BY x1.idNodo, x2.idNodo

```

### 3.2 Arco(id1, id2): quando il DAO deve solo dire chi e collegato

Usala quando la query trova bene la coppia, ma il peso o il verso nasce da un valore del nodo calcolato separatamente.

```

from dataclasses import dataclass

@dataclass
class Arco:
    id1: int
    id2: int

```

Esempi: prodotti orientati per vendite, clienti orientati per spesa, artisti orientati per popolarità, dipendenti orientati per fatturato, generi orientati per fatturato. In questi casi fai una query separata del tipo ID -> valore.

```

# DAO
getAllEdges(...)    -> [Arco(id1, id2)]
getValoriNodo(...)  -> {id: valore}

# Model
val1 = valori[id1]
val2 = valori[id2]
peso = val1 + val2
if val1 > val2: add_edge(n1, n2, weight=peso)
elif val2 > val1: add_edge(n2, n1, weight=peso)
else: add both directions

```

### 3.3 Nessuna dataclass Arco: quando conviene creare archi nel Model

Non creare Arco quando la condizione di arco nasce da dati già salvati dentro i nodi o da set costruiti nel Model. Questo accade spesso nelle prove con richieste complesse sul nodo.

- MediaType acquistati stesso cliente-stesso mese: meglio costruire MediaTypeId -> set(CustomerId, mese) nel Model.
- Playlist simili: puoi caricare PlaylistId -> set(TrackId) e fare intersection nel Model.
- Grafo completo: non serve query archi, basta doppio ciclo sui nodi.
- Circuiti F1 complessi: hai nodi con dizionari di piazzamenti; il peso può derivare da statistiche separate.

## 4. Query dei dropdown: sempre dedicate, sempre semplici

Ogni menu a tendina deve avere una query propria. Queste query sono le più facili, ma se sbagli qui perdi tempo in debug.

```
# Country Chinook
SELECT DISTINCT Country
FROM Customer
WHERE Country IS NOT NULL
ORDER BY Country;

# Anni da una data
SELECT DISTINCT YEAR(InvoiceDate) AS anno
FROM Invoice
ORDER BY anno;

# Generi
SELECT GenreId, Name
FROM Genre
ORDER BY Name;

# Range valori numerici
SELECT DISTINCT Chromosome
FROM genes
WHERE Chromosome IS NOT NULL
ORDER BY Chromosome ASC;
```

### Regola pratica

Il dropdown deve restituire oggetti se poi ti serve un ID e un nome; deve restituire stringhe o interi se il valore stesso basta come filtro.

## 5. Query dei nodi: tradurre "i vertici sono..."

La frase "i vertici sono" è la tua query getAllNodes. La prima decisione è capire se i nodi sono tutti o solo filtrati.

### 5.1 Nodi filtrati con DISTINCT

```
SELECT DISTINCT a.ArtistId, a.Name
FROM Artist a, Album al, Track t
WHERE a.ArtistId = al.ArtistId
      AND al.AlbumId = t.AlbumId
      AND t.GenreId = %s
ORDER BY a.Name;
```

Questo pattern vale quando il nodo è un'entità raggiunta tramite una catena di join: Artist -> Album -> Track -> Genre. DISTINCT evita duplicati perché un artista può avere molte track.

### 5.2 Nodi tutti, anche isolati

```
SELECT *
FROM MediaType
ORDER BY Name;
```

Se il testo dice "i nodi sono tutti i record della tabella MediaType", non devi filtrare in base agli acquisti. Un nodo senza vendite resta isolato. Questo è diverso da "i nodi sono i MediaType acquistati".

### 5.3 Nodi con dati aggiuntivi

Quando il testo dice "per ogni nodo si salvi anche...", il DAO può fare due step: prima crea i nodi, poi riempie liste/dizionari dentro ogni nodo.

```
# Esempio MediaType: struttura nel nodo
MediaType(
  MediaTypeId=1,
  Name='MPEG audio file',
  acquisti_per_mese={
    1: [(CustomerId, InvoiceLineId), ...],
    2: [(CustomerId, InvoiceLineId), ...]
  }
)
```

## 6. Query dizionario valore nodo: ID -> valore

Molte prove orientate hanno una frase come "il verso va dal nodo con valore maggiore al nodo con valore minore". In questi casi crea una funzione DAO dedicata che restituisce un dizionario.

```
def getVenditePerProdotto(...):
    result = {}
    query = """
        SELECT p.product_id,
               COUNT(oi.order_id) AS totale_vendite
        FROM products p, order_items oi, orders o
        WHERE p.product_id = oi.product_id
              AND oi.order_id = o.order_id
              AND p.category_id = %s
              AND o.order_date BETWEEN %s AND %s
        GROUP BY p.product_id
    """
    for row in cursor:
        result[row['product_id']] = int(row['totale_vendite'])
    return result
```

Varianti frequenti:

- COUNT(InvoiceLineId) = numero vendite / popolarità
- SUM(Total) = fatturato o spesa cliente
- SUM(quantity) = quantità totale
- AVG(distance) = distanza media
- COUNT(DISTINCT CustomerId) = clienti distinti

## 7. Query archi non orientati: co-presenza

La maggior parte dei grafi non orientati deriva da una co-presenza: due nodi stanno nella stessa invoice, playlist, exhibition, gara, team, paese, confine.

### 7.1 Pattern base self-join su tabella di associazione

```
SELECT r1.node_id AS id1,
       r2.node_id AS id2,
       COUNT(DISTINCT r1.common_id) AS peso
FROM relazione r1, relazione r2
WHERE r1.common_id = r2.common_id
     AND r1.node_id < r2.node_id
GROUP BY r1.node_id, r2.node_id;
```

Questo schema è perfetto quando hai una tabella ponte: exhibition\_objects, PlaylistTrack, InvoiceLine, role\_mapping, contiguity.

### 7.2 ArtsMia: oggetti nella stessa mostra

```
SELECT eo1.object_id AS id1,
       eo2.object_id AS id2,
       COUNT(*) AS peso
FROM exhibition_objects eo1, exhibition_objects eo2
WHERE eo1.exhibition_id = eo2.exhibition_id
     AND eo1.object_id > eo2.object_id
GROUP BY eo1.object_id, eo2.object_id;
```

Qui Arco(id1,id2,peso) è corretto: l'arco e il peso sono interamente determinati dalla co-presenza nella stessa mostra.

### 7.3 Composer Chinook: compositori nella stessa playlist

```

SELECT t1.Composer AS id1,
       t2.Composer AS id2,
       COUNT(DISTINCT pt1.PlaylistId) AS peso
FROM Track t1, Track t2, PlaylistTrack pt1, PlaylistTrack pt2
WHERE t1.TrackId = pt1.TrackId
      AND t2.TrackId = pt2.TrackId
      AND pt1.PlaylistId = pt2.PlaylistId
      AND t1.Composer IS NOT NULL
      AND t2.Composer IS NOT NULL
      AND t1.Composer <> ''
      AND t2.Composer <> ''
      AND t1.Composer < t2.Composer
      AND t1.MediaTypeId = %s
      AND t2.MediaTypeId = %s
      AND t1.Milliseconds >= %s
      AND t2.Milliseconds >= %s
GROUP BY t1.Composer, t2.Composer;

```

Qui COUNT(DISTINCT PlaylistId) è fondamentale: se nella stessa playlist ci sono più brani dei due compositori, COUNT(\*) sovrastima il peso.

## 7.4 F1 drivers: compagni di squadra

```

SELECT r1.driverId AS id1,
       r2.driverId AS id2,
       COUNT(*) AS peso
FROM results r1, results r2, races ra
WHERE r1.raceId = r2.raceId
      AND r1.constructorId = r2.constructorId
      AND r1.raceId = ra.raceId
      AND ra.year BETWEEN %s AND %s
      AND r1.position IS NOT NULL
      AND r2.position IS NOT NULL
      AND r1.driverId > r2.driverId
GROUP BY r1.driverId, r2.driverId;

```

Qui il peso può essere COUNT(\*) perché ogni riga r1-r2 rappresenta una gara comune nello stesso constructor. Il filtro position IS NOT NULL va su entrambi.

## 7.5 Lab09 aeroporti: LEAST/GREATEST

```

SELECT LEAST(ORIGIN_AIRPORT_ID, DESTINATION_AIRPORT_ID) AS id1,
       GREATEST(ORIGIN_AIRPORT_ID, DESTINATION_AIRPORT_ID) AS id2,
       AVG(DISTANCE) AS peso
FROM flights
GROUP BY id1, id2;

```

Usa LEAST/GREATEST quando il database contiene direzioni A->B e B->A ma il grafo richiesto è non orientato. Così tutte le rotte finiscono nella stessa coppia ordinata.

# 8. Query archi orientati: tre famiglie

## 8.1 Verso temporale: SQL può già orientare

Se il verso è "dal più vecchio al più recente", usa direttamente data1 < data2. Questo crea già id1 sorgente e id2 destinazione.

```

SELECT o1.order_id AS id1,
       o2.order_id AS id2,
       DATEDIFF(o2.order_date, o1.order_date) AS giorni
FROM orders o1, orders o2
WHERE o1.store_id = %s
      AND o2.store_id = %s
      AND o1.order_date < o2.order_date
      AND DATEDIFF(o2.order_date, o1.order_date) <= %s;

```

Se il peso è una formula temporale, puoi calcolarlo in SQL oppure nel Model. Se il testo è complesso ma la formula è chiara, SQL riduce codice Python; se devi studiarla o debuggarla, fai nel Model.

## 8.2 Verso per valore del nodo: meglio Model

Se il verso dice "dal nodo con più vendite/fatturato/popolarità/spesa al minore", di solito conviene: DAO.getAllEdges() per la coppia grezza, DAO.getValoriNodo() per il dizionario, Model orienta.

```
# DAO: coppie grezze
SELECT DISTINCT p1.product_id AS id1, p2.product_id AS id2
FROM products p1, order_items oi1, orders o1,
     products p2, order_items oi2, orders o2
WHERE p1.product_id = oi1.product_id
      AND oi1.order_id = o1.order_id
      AND p2.product_id = oi2.product_id
      AND oi2.order_id = o2.order_id
      AND p1.category_id = %s
      AND p2.category_id = %s
      AND o1.order_date BETWEEN %s AND %s
      AND o2.order_date BETWEEN %s AND %s
      AND p1.product_id > p2.product_id;
```

```
# Model: orientamento
for arco in archi:
    val1 = valori.get(arco.id1, 0)
    val2 = valori.get(arco.id2, 0)
    peso = val1 + val2
    if val1 > val2:
        graph.add_edge(n1, n2, weight=peso)
    elif val2 > val1:
        graph.add_edge(n2, n1, weight=peso)
    else:
        graph.add_edge(n1, n2, weight=peso)
        graph.add_edge(n2, n1, weight=peso)
```

## 8.3 Verso da attributo già nella relazione

Se il verso dipende da un campo già presente nella tabella relazione, puoi lasciare più logica al DAO. Esempio genes: interaction tra GeneID1/GeneID2 con stesso localization, peso Expression\_Corr, verso da cromosoma minore a maggiore. Se il verso richiede guardare entrambi i nodi, anche qui il Model resta più leggibile.

## 9. Pattern temporali

Le prove temporali si riconoscono da parole come: distanza temporale, entro K giorni, data precedente, data successiva, stesso mese, stesso anno.

### 9.1 Entro K giorni

```
AND data1 < data2
AND DATEDIFF(data2, data1) <= %s
```

Usa data1 < data2 per eliminare coppie duplicate e creare il verso cronologico. Se serve escludere stesso giorno, DATEDIFF > 0 oppure data1 < data2 basta se datetime diverso.

### 9.2 Stesso mese / stesso anno

```
AND YEAR(i.InvoiceDate) = %s
AND MONTH(i.InvoiceDate) = %s
```

Se devi confrontare due oggetti nello stesso mese, spesso conviene costruire nel Model un set di coppie (CustomerId, mese).

### 9.3 MediaType Chinook stesso cliente e stesso mese

Qui l'arco non nasce da una semplice query archi; nasce dal confronto fra set per nodo. La traccia chiede anche un dizionario dentro il nodo, quindi è naturale fare la relazione nel Model.

```
# Per ogni MediaType costruisci:
MediaTypeId -> set((CustomerId, mese))

# Due MediaType sono collegati se:
set1.intersection(set2) non è vuoto

# Peso:
vendite1 + vendite2

# Verso:
MediaType con più vendite -> MediaType con meno vendite
```

## 10. Pattern nodi con dati aggiuntivi

### 10.1 Quando il testo dice "per ogni nodo si salvi anche..."

Questa frase significa che la dataclass del nodo deve avere un campo in più: lista o dizionario. Il DAO può riempirlo direttamente, oppure il Model può fare una seconda query per ogni nodo.

```
@dataclass(eq=False)
class Circuit:
    circuitId: int
    name: str
    gare_dettaglio: dict = field(default_factory=dict)

# Nel Model:
for circuito in circuiti:
    dettagli = DAO.get_piazzamenti_circuito(circuito.circuitId, y1, y2)
    circuito.gare_dettaglio = trasforma_dettagli(dettagli)
```

Questo pattern è utile nelle prove tipo 26/06/2025 Formula 1: i nodi sono tutti i circuiti, ma ogni circuito deve sapere quali gare/piazzamenti ha avuto nel range.

### 10.2 Quando il nodo è una stringa ripetuta, non una tabella

Esempio Composer: non esiste tabella Composer. Il nodo va costruito raggruppando Track.Composer.

```
compositori = {}
for row in cursor:
    nome = row['Composer']
    brano = TrackInfo(...)
    if nome not in compositori:
        compositori[nome] = Composer(Name=nome)
    compositori[nome].tracks.append(brano)
return list(compositori.values())
```

## 11. Pattern Chinook principali

### 11.1 Cliente -> Invoice -> InvoiceLine -> Track

Questa è la catena madre per quasi tutte le query Chinook sugli acquisti.

```
FROM Customer c, Invoice i, InvoiceLine il, Track t
WHERE c.CustomerId = i.CustomerId
AND i.InvoiceId = il.InvoiceId
AND il.TrackId = t.TrackId
```

### 11.2 Track -> Album -> Artist

```
FROM Track t, Album al, Artist ar
WHERE t.AlbumId = al.AlbumId
AND al.ArtistId = ar.ArtistId
```

### 11.3 Track -> Genre / MediaType



```
FROM Track t, Genre g, MediaType mt
WHERE t.GenreId = g.GenreId
      AND t.MediaTypeId = mt.MediaTypeId
```

## 11.4 Playlist

```
FROM Playlist p, PlaylistTrack pt, Track t
WHERE p.PlaylistId = pt.PlaylistId
      AND pt.TrackId = t.TrackId
```

## 11.5 Employee supporta Customer

```
FROM Employee e, Customer c
WHERE e.EmployeeId = c.SupportRepId
```

# 12. Casi Chinook svolti: query da ricordare

## 12.1 Artisti per genere - orientato per popolarità

```
-- Nodi
SELECT DISTINCT a.ArtistId, a.Name
FROM Artist a, Album al, Track t
WHERE a.ArtistId = al.ArtistId
      AND al.AlbumId = t.AlbumId
      AND t.GenreId = %s;

-- Valore nodo: popolarità
SELECT al.ArtistId, COUNT(il.InvoiceLineId) AS popolarita
FROM InvoiceLine il, Track t, Album al
WHERE il.TrackId = t.TrackId
      AND t.AlbumId = al.AlbumId
      AND t.GenreId = %s
GROUP BY al.ArtistId;

-- Archi grezzi: cliente ha acquistato entrambi
SELECT c1.ArtistId AS id1, c2.ArtistId AS id2
FROM (... CustomerId, ArtistId ...) c1,
     (... CustomerId, ArtistId ...) c2
WHERE c1.CustomerId = c2.CustomerId
      AND c1.ArtistId < c2.ArtistId;
```

## 12.2 Album co-acquistati - non orientato con peso in SQL

```
SELECT t1.AlbumId AS id1,
       t2.AlbumId AS id2,
       COUNT(DISTINCT il1.InvoiceId) AS peso
FROM InvoiceLine il1, Track t1,
     InvoiceLine il2, Track t2
WHERE il1.TrackId = t1.TrackId
      AND il2.TrackId = t2.TrackId
      AND il1.InvoiceId = il2.InvoiceId
      AND t1.GenreId = %s
      AND t2.GenreId = %s
      AND t1.AlbumId < t2.AlbumId
GROUP BY t1.AlbumId, t2.AlbumId;
```

## 12.3 Invoice temporali di un cliente

Nodi: Invoice del cliente. Arco: distanza temporale entro K. Verso: vecchia -> recente. Peso: (Total1 + Total2) / giorni. Puoi farlo nel Model con nodi ordinati per data: è più leggibile.

```
SELECT InvoiceId, CustomerId, InvoiceDate, Total
FROM Invoice
WHERE CustomerId = %s
ORDER BY InvoiceDate, InvoiceId;
```

## 12.4 Dipendenti e clienti - orientato per fatturato

```

-- Nodi: employee che supportano clienti nel paese
SELECT DISTINCT e.EmployeeId, e.FirstName, e.LastName, e.Title
FROM Employee e, Customer c
WHERE e.EmployeeId = c.SupportRepId
      AND c.Country = %s;

-- Fatturato per employee
SELECT e.EmployeeId, SUM(i.Total) AS fatturato
FROM Employee e, Customer c, Invoice i
WHERE e.EmployeeId = c.SupportRepId
      AND c.CustomerId = i.CustomerId
      AND c.Country = %s
GROUP BY e.EmployeeId;

-- Archi grezzi: generi comuni acquistati dai clienti supportati
SELECT DISTINCT e1.EmployeeId AS id1, e2.EmployeeId AS id2
FROM Employee e1, Customer c1, Invoice i1, InvoiceLine il1, Track t1,
      Employee e2, Customer c2, Invoice i2, InvoiceLine il2, Track t2
WHERE e1.EmployeeId = c1.SupportRepId
      AND c1.CustomerId = i1.CustomerId
      AND i1.InvoiceId = il1.InvoiceId
      AND il1.TrackId = t1.TrackId
      AND e2.EmployeeId = c2.SupportRepId
      AND c2.CustomerId = i2.CustomerId
      AND i2.InvoiceId = il2.InvoiceId
      AND il2.TrackId = t2.TrackId
      AND c1.Country = %s
      AND c2.Country = %s
      AND t1.GenreId = t2.GenreId
      AND e1.EmployeeId < e2.EmployeeId;

```

## 12.5 Composer nelle stesse playlist - non orientato con peso in DAO

Questa è una delle query più utili da ricordare perché mostra quando Arco(id1,id2,peso) è perfetto.

```

SELECT t1.Composer AS id1,
      t2.Composer AS id2,
      COUNT(DISTINCT pt1.PlaylistId) AS peso
FROM Track t1, Track t2, PlaylistTrack pt1, PlaylistTrack pt2
WHERE t1.TrackId = pt1.TrackId
      AND t2.TrackId = pt2.TrackId
      AND pt1.PlaylistId = pt2.PlaylistId
      AND t1.Composer IS NOT NULL
      AND t2.Composer IS NOT NULL
      AND t1.Composer <> ''
      AND t2.Composer <> ''
      AND t1.Composer < t2.Composer
      AND t1.MediaTypeId = %s
      AND t2.MediaTypeId = %s
      AND t1.Milliseconds >= %s
      AND t2.Milliseconds >= %s
GROUP BY t1.Composer, t2.Composer;

```

## 13. Casi repository GitHub: decisione DAO in pratica

### 13.1 2024-07-04 UFO: orientato temporale non pesato

DAO: anni e shape sono dropdown. Nodi: sightings filtrati. Archi: self-join su sighting stessa shape/anno/stato con s1.datetime < s2.datetime. Dataclass Collegamento(id\_passato, id\_futuro). Model: add\_edge senza weight. Componenti: weakly\_connected\_components. Ricorsione: successors, durata crescente, max 3 nello stesso mese, punteggio.

```

SELECT s1.id AS id1, s2.id AS id2
FROM sighting s1, sighting s2
WHERE YEAR(s1.datetime) = %s
      AND YEAR(s2.datetime) = %s
      AND s1.shape = %s
      AND s2.shape = %s
      AND s1.state = s2.state
      AND s1.datetime < s2.datetime;

```

### 13.2 2024-07-18 Genes: range e interactions

DAO: dropdown cromosomi DISTINCT. Nodi: genes con Chromosome BETWEEN. Archi: interactions filtrate su geni nel range, stesso Localization, GeneID diverso. Qui puoi far tornare Interaction completa perché contiene già GeneID1, GeneID2 e Expression\_Corr.

```

SELECT i.*
FROM interactions i, classification c, classification c1
WHERE i.GeneID1 IN (
    SELECT g.GeneID FROM genes g WHERE g.Chromosome BETWEEN %s AND %s
)
AND i.GeneID2 IN (
    SELECT g.GeneID FROM genes g WHERE g.Chromosome BETWEEN %s AND %s
)
AND c.GeneID = i.GeneID1
AND c1.GeneID = i.GeneID2
AND c.Localization = c1.Localization
AND i.GeneID1 != i.GeneID2;

```

### 13.3 2025-11-10 Bike temporal orders

Due versioni possibili: 1) DAO estrae nodi con SUM(quantity) e coppie temporali grezze, Model calcola peso; 2) DAO calcola direttamente peso in SQL. Se il docente dà output basato su query specifica, segui la query del docente.

```

SELECT o1.order_id AS id1,
       o2.order_id AS id2,
       (SUM(o11.quantity) + SUM(o12.quantity)) /
       DATEDIFF(o2.order_date, o1.order_date) AS peso
FROM orders o1, orders o2, order_items oi1, order_items oi2
WHERE o1.store_id = %s
AND o2.store_id = %s
AND o1.order_id = oi1.order_id
AND o2.order_id = oi2.order_id
AND o1.order_date < o2.order_date
AND DATEDIFF(o2.order_date, o1.order_date) <= %s
GROUP BY o1.order_id, o2.order_id;

```

### 13.4 2025-09-15 F1 drivers

Non orientato con peso direttamente in SQL. Dataclass Adiacenza(id1,id2,peso). Ottimo esempio di self-join su results.

### 13.5 2025-07-10 Products

Orientato per vendite: non conviene far orientare SQL. DAO estrae coppie grezze di prodotti venduti e dizionario vendite. Model orienta e pesa.

### 13.6 2025-06-26 F1 circuits

Prova complessa: nodi tutti i circuiti, anche isolati; dettagli piazzamenti per nodo; coppie grezze di circuiti che hanno ospitato nel range; dizionario circuitId -> piloti finiti; Model calcola peso = finiti1 + finiti2. Non forzare una query unica gigantesca.

### 13.7 Lab12 attori

Quando un campo numerico è sporco o testuale, ad esempio incasso con simboli valuta o None, non fare SUM in SQL. DAO restituisce dato grezzo, Model pulisce e aggrega. Questo evita errori MySQL Strict e ti permette di controllare il parsing.

### 13.8 Baseball

Grafo completo non orientato: nessuna query archi. DAO: getAllNodes(anno), getSalariSquadre(anno). Model: doppio ciclo su tutte le squadre e peso = salario1 + salario2.

## 14. Sintassi SQL meccanica: blocchi da copiare

### 14.1 SELECT DISTINCT per nodi unici

```

SELECT DISTINCT n.*
FROM Nodo n, Relazione r
WHERE n.id = r.nodo_id
AND filtro = %s;

```

## 14.2 GROUP BY per valore per nodo

```
SELECT n.id AS id,  
       SUM(x.valore) AS valore  
FROM Nodo n, Relazione r, TabellaValore x  
WHERE n.id = r.nodo_id  
      AND r.id = x.relazione_id  
      AND filtro = %s  
GROUP BY n.id;
```

## 14.3 Self-join per coppie non orientate

```
SELECT DISTINCT a1.id AS id1,  
               a2.id AS id2  
FROM Tabella a1, Tabella a2  
WHERE condizione_comune  
      AND a1.id < a2.id;
```

## 14.4 Self-join per coppie orientate temporali

```
SELECT a1.id AS id1,  
       a2.id AS id2  
FROM Tabella a1, Tabella a2  
WHERE a1.gruppo = a2.gruppo  
      AND a1.data < a2.data;
```

## 14.5 Conteggio distinto per evitare sovrastime

```
COUNT(DISTINCT qualcosa_id) AS peso
```

Usalo quando una stessa entità comune può generare più righe per combinazioni interne: playlist con più brani, invoice con più righe, mostra con più record duplicati, cliente con più acquisti.

## 15. Errori ricorrenti e correzioni rapide

- Errore: usare COUNT(\*) quando la traccia dice numero di entità distinte. Correzione: COUNT(DISTINCT id).
- Errore: filtrare i nodi sulle vendite quando il testo dice tutti i prodotti/tutti i MediaType/tutti i circuiti. Correzione: aggiungi tutti i nodi e lascia isolati quelli senza archi.
- Errore: creare due archi in grafo non orientato. Correzione: usa id1 < id2 o id1 > id2.
- Errore: usare successors() su grafo non orientato. Correzione: usa neighbors().
- Errore: fare orientamento in SQL quando dipende da un dizionario di valori. Correzione: DAO coppie grezze + dizionario; Model orienta.
- Errore: dimenticare stesso filtro su entrambi i lati del self-join. Correzione: t1.MediaTypeId=%s e t2.MediaTypeId=%s, date su o1 e o2, anno su r1 e r2.
- Errore: confrontare date in stringa nel Model se il DB restituisce datetime. Correzione: salva Date/DateTime nella dataclass e sottrai direttamente.
- Errore: usare valori fissi in DAO. Correzione: nel DAO sempre %s, i valori fissi solo in DBeaver per test.

## 16. Checklist finale prima di scrivere codice

| Domanda               | Risposta che devi avere prima di codificare                            |
|-----------------------|------------------------------------------------------------------------|
| Dropdown?             | Una funzione DAO per ogni dropdown.                                    |
| Nodi?                 | Query getAllNodes con SELECT DISTINCT se serve.                        |
| Nodi isolati?         | Se sì, non filtrare i nodi sulla relazione di arco.                    |
| Dati extra nel nodo?  | Aggiungi campo lista/dict nella dataclass e popola con query dedicata. |
| Valore per orientare? | Crea dizionario ID -> valore.                                          |

| Domanda         | Risposta che devi avere prima di codificare                          |
|-----------------|----------------------------------------------------------------------|
| Arco in SQL?    | Sì se peso/relazione sono aggregate facilmente con GROUP BY.         |
| Arco grezzo?    | Sì se peso/verso dipendono da valori nodo.                           |
| Arco nel Model? | Sì se serve confrontare set/list/dizionari o se il grafo è completo. |
| Orientato?      | DiGraph + successors; verso da data, valore o attributo.             |
| Non orientato?  | Graph + neighbors; id1 < id2 o LEAST/GREATEST.                       |
| Peso?           | Scrivi la formula prima in italiano, poi in SQL/Python.              |
| Ricorsione?     | parziale, best, vincoli, append, recursive call, pop.                |

## 17. Mini-scheletro DAO ideale

```
class DAO:

    @staticmethod
    def getAllDropdownValues():
        # SELECT DISTINCT ... ORDER BY ...
        return lista

    @staticmethod
    def getAllNodes(parametri):
        # SELECT DISTINCT nodi ... WHERE filtri
        return lista_oggetti_nodo

    @staticmethod
    def getValoriNodo(parametri):
        # SELECT id, SUM/COUNT/AVG(...) GROUP BY id
        return dizionario_id_valore

    @staticmethod
    def getAllEdges(parametri):
        # Se conviene: SELECT id1, id2, peso con self-join/GROUP BY
        # Oppure: SELECT DISTINCT id1, id2 per archi grezzi
        return lista_archi
```

## 18. Mini-scheletro Model ideale

```
def buildGraph(self, parametri):
    self._graph.clear()
    self._idMap.clear()

    nodi = DAO.getAllNodes(parametri)
    for n in nodi:
        self._graph.add_node(n)
        self._idMap[n.id] = n

    # Opzione A: archi già pesati
    for arco in DAO.getAllEdges(parametri):
        n1 = self._idMap[arco.id1]
        n2 = self._idMap[arco.id2]
        self._graph.add_edge(n1, n2, weight=arco.peso)

    # Opzione B: archi grezzi + valori
    valori = DAO.getValoriNodo(parametri)
    for arco in archi_grezzi:
        val1 = valori.get(arco.id1, 0)
        val2 = valori.get(arco.id2, 0)
        peso = val1 + val2
        # orienta qui

    return self._graph.number_of_nodes(), self._graph.number_of_edges()
```

## 19. Regola conclusiva: scegli la soluzione più stabile in esame

In esame non vince la query più elegante, vince la soluzione corretta, leggibile e debuggabile. Se una query SQL diventa troppo lunga e rischi di perderti, spezzala: query nodi, query dizionario valori, query archi grezzi. Se invece il peso è un COUNT DISTINCT naturale per coppia, calcolalo direttamente nel DAO con Arco(id1,id2,peso).

### Frase da ricordare

SQL deve estrarre bene. Il Model deve ragionare bene. Non obbligare SQL a ragionare se il ragionamento è più chiaro con un dizionario Python.

## 20. Riferimenti URL dei repository considerati

[https://github.com/Samu-TDP/2024-07-04-A-grafo\\_orientato](https://github.com/Samu-TDP/2024-07-04-A-grafo_orientato)

[https://github.com/Samu-TDP/2024-07-18-A\\_grafo\\_orientato](https://github.com/Samu-TDP/2024-07-18-A_grafo_orientato)

[https://github.com/Samu-TDP/2025-11-10\\_grafo\\_orientato](https://github.com/Samu-TDP/2025-11-10_grafo_orientato)

[https://github.com/Samu-TDP/2025-09-15-A\\_grafo\\_non\\_orientato](https://github.com/Samu-TDP/2025-09-15-A_grafo_non_orientato)

[https://github.com/Samu-TDP/2025-07-10-A\\_grafo\\_orientato](https://github.com/Samu-TDP/2025-07-10-A_grafo_orientato)

[https://github.com/Samu-TDP/2025-06-26-A\\_grafo\\_non\\_orientato](https://github.com/Samu-TDP/2025-06-26-A_grafo_non_orientato)

[https://github.com/Samu-TDP/Lab11-Simulazione-Esime\\_Database29Giugno\\_grafo\\_orientato](https://github.com/Samu-TDP/Lab11-Simulazione-Esime_Database29Giugno_grafo_orientato)

[https://github.com/Samu-TDP/Lab12-Simulazione-esame\\_grafo\\_non\\_orientato](https://github.com/Samu-TDP/Lab12-Simulazione-esame_grafo_non_orientato)

[https://github.com/Samu-TDP/baseball\\_grafo\\_non\\_orientato](https://github.com/Samu-TDP/baseball_grafo_non_orientato)

[https://github.com/Samu-TDP/ArtsMia\\_grafo\\_non\\_orientato](https://github.com/Samu-TDP/ArtsMia_grafo_non_orientato)

[https://github.com/Samu-TDP/Lab10\\_grafo\\_non\\_orientato](https://github.com/Samu-TDP/Lab10_grafo_non_orientato)

[https://github.com/Samu-TDP/Lab09\\_grafo\\_non\\_orientato](https://github.com/Samu-TDP/Lab09_grafo_non_orientato)